Glava 1

Radno okruženje i merenje performansi

Cilj vežbe

- Utvrditi CPU, kompajler i uslove merenja na radnom računaru.
- Izabrati granice merenja i razlikovati kašnjenje, propusnost i ubrzanje.
- Realizovati dve sekvencijalne verzije pokretne srednje vrednosti i proveriti ispravnost.
- Predvideti uticaj veličine ulaza i prozora, a zatim proveriti predviđanje merenjem.

Sve komande u ovoj vežbi pokreću se iz `root_dir`, direktorijuma raspakovane arhive `pml.zip`. Uz svako merenje navesti radni računar, CPU, verziju kompajlera, opcije kompajliranja, veličinu ulaza, broj ponavljanja, baznu verziju i metriku sa jedinicom.

1.1 Provera okruženja

```
lscpu
g++ --version
nproc
getconf LEVEL1_DCACHE_LINESIZE
```

Razlikovati broj fizičkih jezgara od broja logičkih CPU niti. Zabeležiti da li se merenje izvodi na fizičkoj mašini ili u virtuelnom okruženju. Zabeležiti raspoloživu RAM memoriju komandom `free -h`.

1.2 Praćenje procesa

Kratko posmatrati jedno pokretanje programa. Dodatne komande ispod služe kao podsetnik kada treba proveriti opterećenje ili potrošnju memorije.

U jednom terminalu pokrenuti dovoljno dug proračun, a u drugom pratiti opterećenje mašine. Za početak se može koristiti sekvencijalni primer:

```
g++ -O3 -Wall -Wextra \
    src/bench_seq.cpp -o src/bench_seq
./src/bench_seq 20000000
```

U drugom terminalu pokrenuti alat za praćenje:

```
top
pgrep -x bench_seq
```

Pritiskom na taster `1` u programu `top` prikazuje se opterećenje svakog logičkog CPU-a, a taster `H` prebacuje prikaz na pojedinačne niti. Taster `q` završava praćenje. Pronaći PID programa i zabeležiti njegovo CPU opterećenje, stanje i zauzeće radne memorije (kolona RES). Ova kolona

pokazuje koliko memorije procesa se trenutno nalazi u RAM-u. U uobičajenom prikazu 100% znači da proces potpuno koristi jedan logički CPU. Na primer, 200% znači približno dva tako zauzeta CPU-a; višenitni proces zato može preći 100%. Alat `top` ovaj način brojanja naziva Irix režimom. Taster `I` menja prikaz: u drugom režimu procenat se deli brojem CPU-a. Proveriti izabrani režim pre tumačenja procenta. Kratak program može da se završi između dva osvežavanja: tada povećati ulaz, vodeći računa o raspoloživoj memoriji.

1.2.1 Dodatne komande za praćenje

Za poznati PID, na primer 12345, mogu se koristiti:

```
top -H -p 12345
ps -L -p 12345 -o pid,tid,psr,pcpu,stat,comm
command time -v ./src/bench_seq 20000000
```

Broj 12345 zameniti stvarnim PID-om. U Bash-u zapis `command time` bira spoljašnji GNU alat, koji podržava opciju `-v`; sama reč `time` ima i posebno značenje u komandnom okruženju. Kolona TID označava OS nit, a PSR logički CPU na kojem se ona poslednji put izvršavala. Operativni sistem može premestiti nit na drugi logički CPU tokom rada. Sekvencijalni primer ima jednu programsku nit; isti postupak može da se primeni na sopstveni višenitni program. Spoljašnji alat za vreme obuhvata ceo proces, uključujući alokaciju i ispis; njegov rezultat se razlikuje od vremena samog proračuna u programu. Zabeležiti i broj promena konteksta i najveće zauzeće radne memorije tokom izvršavanja (`Maximum resident set size` u ispisu alata). Posmatranje alatima obaviti odvojeno od serije merenja performansi.

1.3 Kako meriti

Ceo program je u `src/bench_seq.cpp`. Za granice merenja važan je ovaj deo:

```
const auto t0 = chrono::steady_clock::now();
double sum = 0.0;
for (size_t i = 0; i < n; ++i)
{
    sum += sin(a[i]) * sin(a[i]) + cos(a[i]) * cos(a[i]);
}
const auto t1 = chrono::steady_clock::now();

const double elapsed = chrono::duration<double>(t1 - t0).count();
```

```
g++ -O3 -Wall -Wextra \
    src/bench_seq.cpp -o src/bench_seq
./src/bench_seq 1000000
```

Časovnik `steady_clock` služi za merenje proteklog vremena. Njegove vrednosti ne idu unazad kada se podesi datum ili sinhronizuje sistemsko vreme. Zato je razlika dva čitanja pogodna za trajanje proračuna. Za razliku od njega, časovnik za datum i vreme može biti korigovan tokom rada. Izmereno trajanje uključuje i vreme u kojem proces čeka da ponovo dobije CPU.

Identifikovati tačno koje naredbe su između dva čitanja časovnika. Da li su alokacija, popunjavanje ulaza i ispis uključeni u prijavljeno vreme? Objasniti zašto se izračunati rezultat ispisuje. Proveriti očekivani rezultat koristeći $\sin^2(a) + \cos^2(a) = 1$, uz moguće male greške računanja u pokretnom zarezu.

Za isti ulaz pokrenuti program jednom za zagrevanje, a zatim još pet puta. Sačuvati svih pet *merenih vremena* i izračunati njihovu aritmetičku sredinu. Navesti i najmanje i najveće mereno vreme. Ne odbacivati neobično vreme bez objašnjenja; ako se eksperiment ponovi, sačuvati i prethodna merenja.

Kašnjenje jednog izvršenja je vreme od početka do završetka posmatranog proračuna. Propusnost opisuje količinu završenog rada po jedinici vremena. Nisu ista metrika, iako se za fiksnu količinu rada mogu povezati.

1.4 Otklanjanje grešaka

Za otklanjanje grešaka napraviti poseban izvršni fajl sa simbolima i malim stepenom optimizacije. Na njemu ne meriti performanse:

```
g++ -g -Og src/bench_seq.cpp -o /tmp/bench_debug
gdb --args /tmp/bench_debug 1000
```

Sledeće komande su za GDB, alat za otklanjanje grešaka. Ako koristite odgovarajući alat u svom IDE-u, uradite isto: postavite tačku prekida, pratite izvršavanje korak po korak i pogledajte promenljive i stek poziva. U GDB-u zadati:

```
start
next
print n
list
backtrace
continue
quit
```

Komanda za početak zaustavlja program u funkciji `main`. Jedan korak prelazi preko inicijalizacije veličine ulaza, pa se njena vrednost može ispisati. Prikaz steka pokazuje lanac aktivnih poziva. Tačka prekida postavlja se komandom `break`, uz ime funkcije ili fajl i broj reda.

1.5 Pokretna srednja vrednost

Ulaz je niz x sa N uzoraka. Za prozor od W uzoraka izračunati niz y:

$$y_i = \frac{1}{W} \sum_{j=0}^{W-1} x_{i+j}, \qquad 0 \le i < N - W + 1, \qquad 1 \le W \le N.$$

Koristiti tip `double`. Izlaz ima $M = N - W + 1$ elemenata; računaju se samo potpuni prozori, bez dopunjavanja ulaza nulama.

1.5.1 Dve realizacije

Dopuniti dve funkcije u fajlu `src/average/sequential.cpp`. Program `src/moving_average.cpp` već priprema ulaz i izlaz, proverava rezultat i meri vreme. Dopunjujete proračun i svoje male testove:

```
void moving_average_direct(const Samples& x, int w,
    Samples& y);
void moving_average_sliding(const Samples& x, int w,
    Samples& y);
```

Tip `Samples` je niz brojeva tipa `double`, zapisan kao `vector<double>`. Za indekse i veličinu prozora koristiti `int`; zadate veličine su dovoljno male za taj tip. U polaznom kodu parametri su neimenovani; dodati imena `x`, `w` i `y`. Pozive koji prijavljuju nezavršenu funkciju zameniti sopstvenim proračunom.

1. **Direktna verzija:** za svaki izlazni element nezavisno sabrati svih W uzoraka i podeliti sumu sa W.

2. **Verzija sa ponovnom upotrebom sume:** izračunati sumu prvog prozora. Pri pomeranju prozora oduzeti uzorak koji izlazi i dodati uzorak koji ulazi. Srednju vrednost izračunati iz ažurirane sume.

Svaka funkcija mora upisati sve elemente već alociranog niza `y`. U funkcijama ne alocirati memoriju, ne ispisivati i ne meriti vreme. Polazna funkcija šalje poruku o grešci kroz `cerr` i završava program pozivom `exit(1)` dok TODO zadatak nije urađen.

1.5.2 Šta se menja pri pomeranju prozora

Za ulaz $[1, 2, 3, 4, 5]$ i $W = 3$, prvi prozor je $[1, 2, 3]$. Njegova suma je 6, pa je prvi izlaz $6/3 = 2$. Sledeći prozor je $[2, 3, 4]$. Direktni postupak ponovo sabira sva tri broja i dobija 9. Klizni postupak zadržava prethodnu sumu: izbacuje 1 i dodaje 4, pa takođe dobija $6 - 1 + 4 = 9$, odnosno srednju vrednost 3.

Ako je suma prozora koji počinje u i označena sa s_i, onda važi

$$s_{i+1} = s_i - x_i + x_{i+W}, \qquad y_{i+1} = s_{i+1}/W.$$

Čuva se *suma*, a deljenje daje izlaz. Prvi prozor se u oba postupka sabira ceo; tek sledeći prozori mogu da iskoriste prethodni rezultat. Direktni postupak za svaki izlaz ima W sabiranja. Klizni posle prvog prozora ima jedno oduzimanje i jedno sabiranje, bez obzira na W. Zato veći prozor povećava rad direktnog postupka mnogo više. Direktni prozori su nezavisni, dok klizno ažuriranje koristi prethodnu sumu. To će biti važno kada posao budemo delili nitima.

1.5.3 Provera ispravnosti pre merenja

Za ulaz $[1, 2, 3, 4, 5, 6, 7, 8]$ i $W = 3$ očekivani izlaz je $[2, 3, 4, 5, 6, 7]$. Ručno izračunati prva dva izlazna elementa. Proveriti i $W = 1$ (izlaz jednak ulazu) i $W = N$ (jedan izlazni element, 4.5). Zatim proveriti ulaz sa jednim uzorkom i dodati sopstveni mali test sa nejednakim, uključujući negativne, vrednostima. Za $W = 0$ ili $W > N$ program mora prijaviti grešku bez merenja.

Polazni program poredi svaki element. Tip `double` čuva brojeve sa ograničenom preciznošću, pa se rezultati operacija zaokružuju. Direktna verzija svaki prozor sabira iz početka, a druga ažurira prethodnu sumu oduzimanjem i dodavanjem. Zato rezultati mogu malo da se razlikuju i kada su obe realizacije ispravne. Umesto tačne jednakosti, provera dozvoljava malu razliku:

$$|y_i - y_i^{\mathrm{ref}}| \le 10^{-8} \max(1, |y_i^{\mathrm{ref}}|).$$

Ovde je y_i^{ref} referentna vrednost. Za njenu apsolutnu vrednost do 1 dozvoljena razlika je 10^{-8}; za veće vrednosti granica raste srazmerno njihovoj veličini. Ovo je prag provere za zadate ulaze, a ne garancija tačnosti za svaki mogući ulaz. Neslaganje iznad tolerancije ili nekonačna vrednost zaustavlja eksperiment; vreme neispravne realizacije ne koristiti za poređenje.

```
g++ -O3 -Wall -Wextra src/moving_average.cpp \
    src/average/sequential.cpp -o src/moving_average
./src/moving_average --check
./src/moving_average 8 3
./src/moving_average 8 0
./src/moving_average 8 9
```

Poslednja dva poziva treba da prijave grešku. Uspešan `-check` proverava obe verzije za tri zadata mala slučaja; sopstvene testove dodati u funkciju `check_small_inputs` u fajlu `src/moving_average.cpp`.

1.6 Predviđanje i plan eksperimenta

Pre merenja odgovoriti:

1. Koliko sabiranja i deljenja ima direktna verzija za M izlaznih elemenata? Koliko sabiranja, oduzimanja i deljenja ima druga verzija?

2. Šta očekujete kada se N poveća deset puta za isto W?

3. Šta očekujete kada se W poveća sa 8 na 64 za isto N?

4. Da li broj aritmetičkih operacija sam određuje vreme izvršavanja? Navedite još jedan mogući faktor.

Uporediti obe verzije za svih šest kombinacija:

$$N \in \{100000, 1000000, 5000000\}, \qquad W \in \{8, 64\}.$$

Polazni program generiše isti ulaz za obe verzije: `x[i] = 0.001 * (i % 1000)`. Za svaku kombinaciju izvršava jedno zagrevanje po verziji i pet merenih ponavljanja. Redosled izvršavanja verzija se smenjuje. Zagrevanje se ne uključuje u srednju vrednost.

Između dva čitanja časovnika nalazi se samo poziv funkcije za proračun. Alokacija, priprema ulaza i izlaza, provera, kontrolna suma i ispis su van merenog intervala. Ovo je *vreme proračuna*. Svaki rezultat se koristi posle merenja. Ne uklanjati provere radi boljeg vremena.

1.6.1 Opcije optimizacije

Opcije `-O2` i `-O3` biraju nivo optimizacije kompajlera. Opcija `-O2` uključuje širok skup postupaka koji pokušavaju da ubrzaju program. Opcija `-O3` dodaje još transformacija, naročito petlji, i može proizvesti veći mašinski kod. Veći broj ne garantuje kraće vreme: rezultat zavisi od programa i procesora, pa ga treba izmeriti.

Za merenje koristimo `-O3`; podrazumevane opcije su `-O3 -Wall -Wextra`. Poslednje dve uključuju upozorenja kompajlera. Za praćenje programa u GDB-u koristimo `-g -Og`: simbole za prepoznavanje izvora i optimizaciju prilagođenu otklanjanju grešaka. Verzije koje poredimo kompajlirati istim opcijama i sačuvati komandu. Na primer, zaseban eksperiment sa `-O2` može se pripremiti ovako:

```
g++ -O2 -Wall -Wextra src/moving_average.cpp \
    src/average/sequential.cpp -o src/moving_average
```

Skripta za merenje ponovo kompajlira program. Ako i nju koristite za taj eksperiment, proslediti iste opcije, na primer:

```
CXXFLAGS="-O2 -Wall -Wextra" python3 scripts/run_experiments.py \
    1 --out results/lab1_O2
```

Ne koristiti `-ffast-math` u ovoj vežbi i ne pokretati druge eksperimente istovremeno. Detalji su u GCC uputstvu za optimizaciju.

Za jednu kombinaciju:

```
./src/moving_average 100000 8 measured_100000_8.csv \
    > summary_100000_8.txt
```

Poslednji argument zadaje CSV fajl u koji program upisuje merenja. Kada se ime izostavi, koristi se `moving_average.csv`. U kodu objekat `ofstream` otvara fajl, a `csv «` upisuje podatke u njega. Provere i sažetak ispisuju se pomoću `cout` na konzolu; znak `>` u prikazanoj komandi čuva taj ispis u tekstualnom fajlu. Poruke o greškama idu kroz `cerr`. Za svako novo merenje zadati novo ime fajla. Ponoviti za preostalih pet kombinacija. CSV sadrži sledeće kolone:

```
method,n,w,repetition,time_s,checksum
```

Izveštaj sadrži proveru, srednju vrednost, minimum, maksimum i propusnost. Sačuvati oba fajla. Proveriti status programa i poruke o ispravnosti pre analize CSV-a.

Za svaku verziju prikazati propusnost $Q = M/\overline{T}$ u izlaznim elementima u sekundi. Relativno ubrzanje druge verzije je

$$U = \frac{\overline{T}_{\text{direct}}}{\overline{T}_{\text{sliding}}}.$$

Ovo je poređenje dva sekvencijalna algoritma; paralelna efikasnost se ovde ne računa.

1.7 Tumačenje i samostalna provera

Napraviti tabelu sa svih pet vremena, srednjom vrednošću, minimumom i maksimumom za svaku verziju i kombinaciju (N, W). Nacrtati vreme u funkciji N; za svako W prikazati obe verzije na istom grafiku. Označiti ose i jedinice. Uporediti predviđanja sa rezultatima, posebno zavisnost od W. Teorijski broj pristupa elementima ne predstavljati kao izmeren saobraćaj memorije ili broj keš promašaja.

U poslednjem delu vežbe nastavnik zadaje novu veličinu prozora, na primer $W = 1$, $W = 17$ ili $W = 128$, za jednu već merenu veličinu ulaza. Samostalno predvideti uticaj na obe verzije, pokrenuti proveru i merenje, a zatim objasniti rezultat. Nastavniku pokazati gde su granice merenja i koji indeksi označavaju uzorak koji izlazi i uzorak koji ulazi.

Za prikupljanje svih šest kombinacija i crtanje koristiti priložene skripte:

```
python3 scripts/run_experiments.py 1 --out results/lab1
python3 scripts/plot_results.py results/lab1
```

Skripta ponovo kompajlira i proverava program, pa čuva komandu kompajliranja, podatke o mašini i sva vremena. Za novu seriju zadati novi direktorijum. Za crtanje je potreban Python paket Matplotlib. Tabela `summary.csv` i dva grafika su polazna tačka za tumačenje; objasniti šta prikazuje svaka kriva. Zabeležiti i da li je računar virtuelan i da li je bio opterećen.

Rezultat vežbe

- Pokazati sopstvene dve funkcije, mali test i granice merenja.
- Pokazati sačuvana vremena za šest kombinacija i dva grafika. Objasniti jednu razliku između predviđanja i merenja.
- Predvideti i proveriti rezultat za novu veličinu prozora.